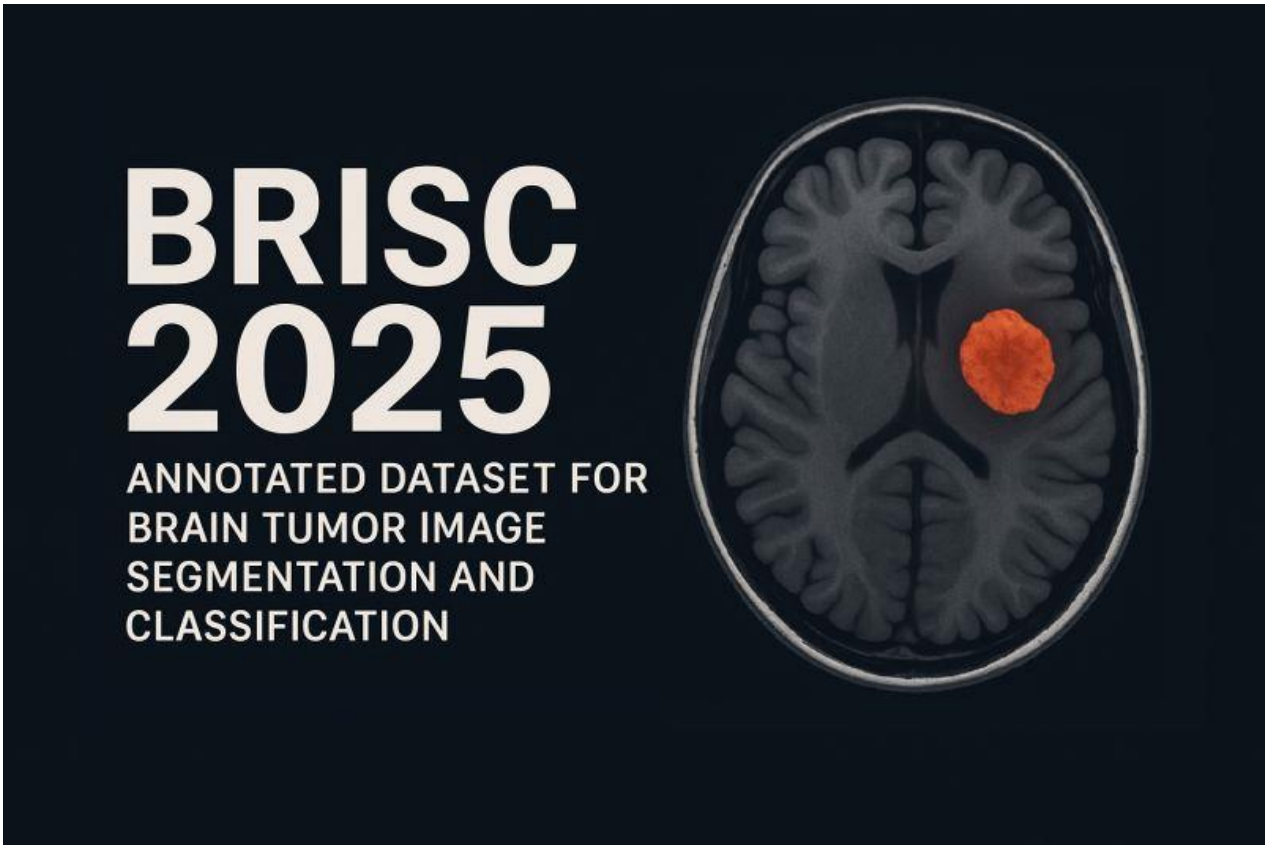
Brain Tumor Classification with ArConvNet




```
import numpy as np
import pandas as pd
import os

root_path = '/kaggle/input/brisc2025/brisc2025/classification_task/train'

image_paths = []
labels = []

for label in os.listdir(root_path):
    label_path = os.path.join(root_path, label)
    if os.path.isdir(label_path):
        for img_file in os.listdir(label_path):
            if img_file.lower().endswith(('.png', '.jpg', '.jpeg')):
                image_paths.append(os.path.join(label_path, img_file))
                labels.append(label)

df = pd.DataFrame({'image_path': image_paths, 'label': labels})

df
```

```
                                              image_path      label
0     /kaggle/input/brisc2025/brisc2025/classificati...  pituitary
1     /kaggle/input/brisc2025/brisc2025/classificati...  pituitary
2     /kaggle/input/brisc2025/brisc2025/classificati...  pituitary
3     /kaggle/input/brisc2025/brisc2025/classificati...  pituitary
4     /kaggle/input/brisc2025/brisc2025/classificati...  pituitary
...                                                 ...        ...
4995  /kaggle/input/brisc2025/brisc2025/classificati...     glioma
4996  /kaggle/input/brisc2025/brisc2025/classificati...     glioma
4997  /kaggle/input/brisc2025/brisc2025/classificati...     glioma
4998  /kaggle/input/brisc2025/brisc2025/classificati...     glioma
4999  /kaggle/input/brisc2025/brisc2025/classificati...     glioma

[5000 rows x 2 columns]
```

```
df.shape
```

```
(5000, 2)
```

```
df.columns
```

```
Index(['image_path', 'label'], dtype='object')
```

```
df.duplicated().sum()
```

```
0
```

```
df.isnull().sum()
```

```
image_path    0
label         0
dtype: int64
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5000 entries, 0 to 4999
Data columns (total 2 columns):
 #   Column      Non-Null Count  Dtype 
---  ------      --------------  ----- 
 0   image_path  5000 non-null   object
 1   label       5000 non-null   object
dtypes: object(2)
memory usage: 78.3+ KB
```

```
df['label'].unique()
```

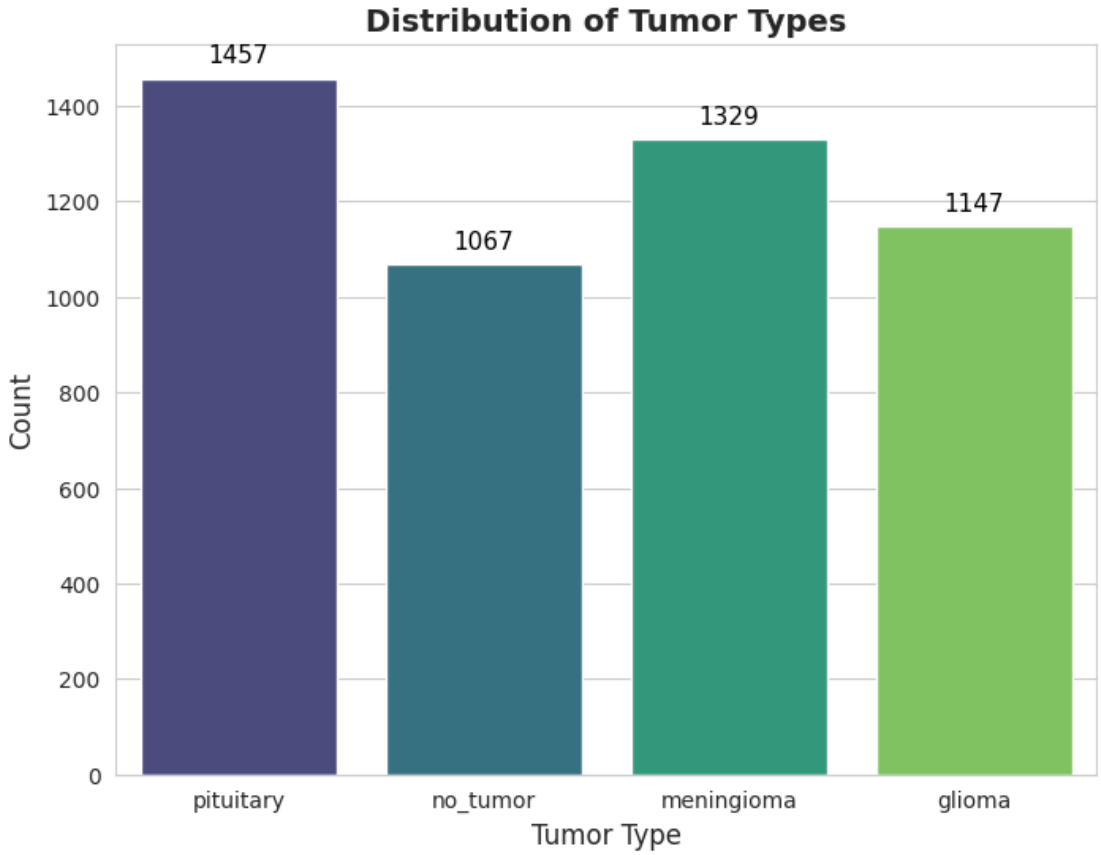
```
array(['pituitary', 'no_tumor', 'meningioma', 'glioma'], dtype=object)
```

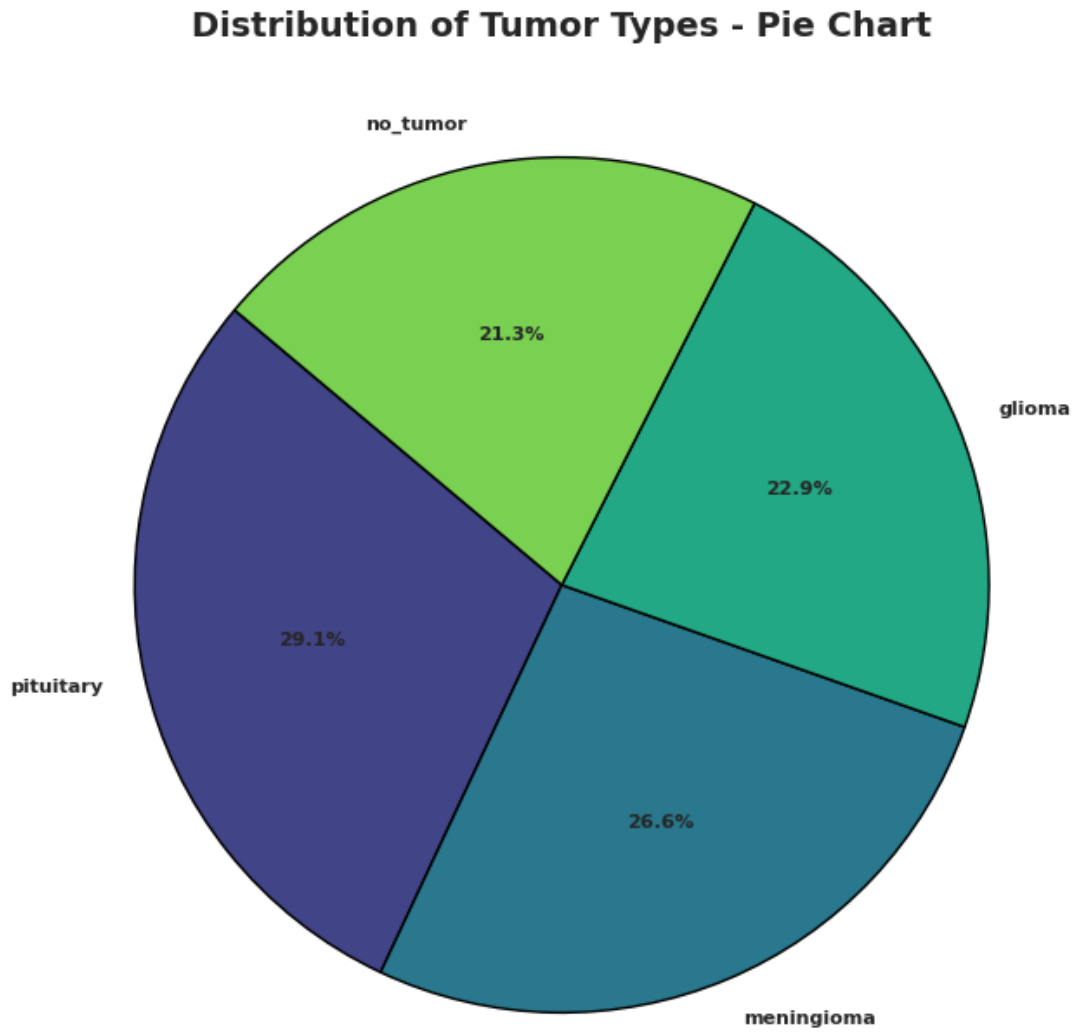
```
df['label'].value_counts()
```

```
label
pituitary     1457
```

```
meningioma    1329
glioma        1147
no_tumor      1067
Name: count, dtype: int64
```

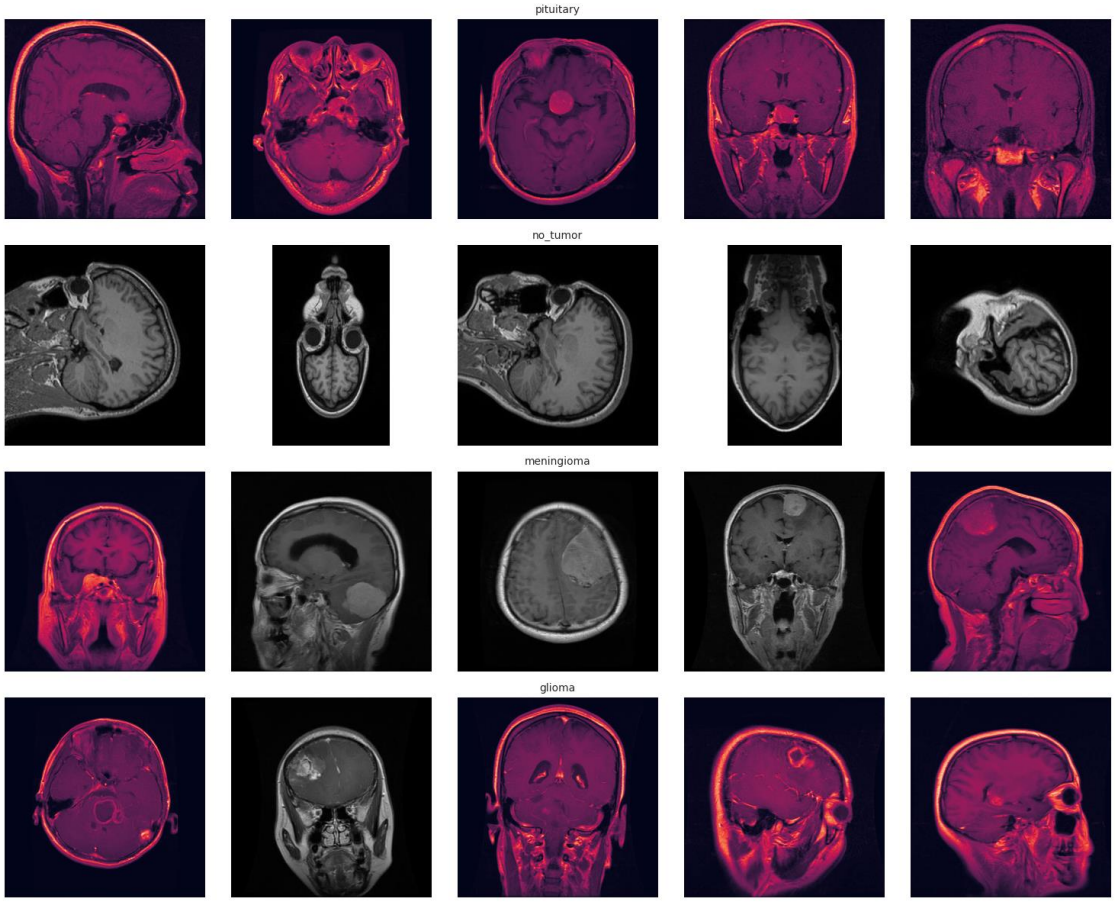
```
import seaborn as sns
import matplotlib.pyplot as plt

sns.set_style("whitegrid")

fig, ax = plt.subplots(figsize=(8, 6))
sns.countplot(data=df, x="label", palette="viridis", ax=ax)

ax.set_title("Distribution of Tumor Types", fontsize=14, fontweight='bold')
ax.set_xlabel("Tumor Type", fontsize=12)
ax.set_ylabel("Count", fontsize=12)

for p in ax.patches:
    ax.annotate(f'{int(p.get_height())}',
                (p.get_x() + p.get_width() / 2., p.get_height()),
                ha='center', va='bottom', fontsize=11, color='black',
                xytext=(0, 5), textcoords='offset points')

plt.show()

label_counts = df["label"].value_counts()

fig, ax = plt.subplots(figsize=(10, 8))
colors = sns.color_palette("viridis", len(label_counts))

ax.pie(label_counts, labels=label_counts.index, autopct='%1.1f%%',
       startangle=140, colors=colors, textprops={'fontsize': 8, 'weight':
'bold'},
       wedgeprops={'edgecolor': 'black', 'linewidth': 1})

ax.set_title("Distribution of Tumor Types - Pie Chart", fontsize=14,
fontweight='bold')

plt.show()
```


Distribution of Tumor Types
1457
1329
1147
1067
1400
1200
1000
800
600
400
200
0
Count
pituitary
no_tumor
meningioma
glioma
Tumor Type




```
from PIL import Image

num_images = 5

unique_labels = df['label'].unique()

plt.figure(figsize=(15, len(unique_labels) * 3))

for row_idx, label in enumerate(unique_labels):

    label_images = df[df['label'] ==
label].head(num_images)['image_path'].tolist()

    for col_idx, img_path in enumerate(label_images):
        plt_idx = row_idx * num_images + col_idx + 1
```

```
        plt.subplot(len(unique_labels), num_images, plt_idx)
        img = Image.open(img_path)
        plt.imshow(img)
        plt.axis('off')
        if col_idx == 2:
            plt.title(label, fontsize=10)

plt.tight_layout()
plt.show()
```




```
import warnings
warnings.filterwarnings('ignore')

max_samples = df['label'].value_counts().max()

balanced_df = df.groupby('label', group_keys=False).apply(
    lambda x: x.sample(n=max_samples, replace=True, random_state=42)
).reset_index(drop=True)

balanced_df = balanced_df[['image_path', 'label']]

df = balanced_df
```

```
df
```

```
                                             image_path      label
0     /kaggle/input/brisc2025/brisc2025/classificati...     glioma
1     /kaggle/input/brisc2025/brisc2025/classificati...     glioma
2     /kaggle/input/brisc2025/brisc2025/classificati...     glioma
3     /kaggle/input/brisc2025/brisc2025/classificati...     glioma
4     /kaggle/input/brisc2025/brisc2025/classificati...     glioma
...                                                 ...        ...
5823  /kaggle/input/brisc2025/brisc2025/classificati...  pituitary
5824  /kaggle/input/brisc2025/brisc2025/classificati...  pituitary
5825  /kaggle/input/brisc2025/brisc2025/classificati...  pituitary
5826  /kaggle/input/brisc2025/brisc2025/classificati...  pituitary
5827  /kaggle/input/brisc2025/brisc2025/classificati...  pituitary

[5828 rows x 2 columns]
```

```
import torch
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim
from torchvision import transforms
from torch.utils.data import Dataset, DataLoader
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.metrics import confusion_matrix, classification_report
import matplotlib.pyplot as plt
import seaborn as sns
import os
from PIL import Image

class ArConv(nn.Module):
    def __init__(self, channels, kernel_size=3, stride=1, padding=1):
        super(ArConv, self).__init__()
        self.dw_conv = nn.Conv2d(channels, channels, kernel_size=(1,
kernel_size), stride=(1, stride), padding=(0, padding), groups=channels,
bias=False)

    def forward(self, x):
        y = self.dw_conv(x)
        y = y.permute(0, 1, 3, 2)
        y = self.dw_conv(y)
        y = y.permute(0, 1, 3, 2)
        return y

class ArConvBlock(nn.Module):
    def __init__(self, in_channels, out_channels, expansion, stride=1):
        super(ArConvBlock, self).__init__()
        exp_channels = in_channels * expansion
```

```
        self.expand = nn.Conv2d(in_channels, exp_channels, kernel_size=1,
bias=False)
        self.bn1 = nn.BatchNorm2d(exp_channels)
        if stride == 2:
            self.dw = nn.Conv2d(exp_channels, exp_channels, kernel_size=3,
stride=2, padding=1, groups=exp_channels, bias=False)
        else:
            self.dw = ArConv(exp_channels, kernel_size=3, stride=stride,
padding=1)
        self.bn2 = nn.BatchNorm2d(exp_channels)
        self.project = nn.Conv2d(exp_channels, out_channels, kernel_size=1,
bias=False)
        self.bn3 = nn.BatchNorm2d(out_channels)
        self.stride = stride
        self.equal_dim = (in_channels == out_channels and stride == 1)

    def forward(self, x):
        identity = x
        out = self.expand(x)
        out = self.bn1(out)
        out = F.relu(out)
        out = self.dw(out)
        out = self.bn2(out)
        out = F.relu(out)
        out = self.project(out)
        out = self.bn3(out)
        if self.equal_dim:
            out = out + identity
        return out

class ArConvNet(nn.Module):
    def __init__(self, num_classes=4):
        super(ArConvNet, self).__init__()
        self.initial = nn.Conv2d(3, 24, kernel_size=4, stride=4, padding=0)
        self.blocks = nn.Sequential(
            ArConvBlock(24, 24, 4, 1),
            ArConvBlock(24, 24, 4, 1),
            ArConvBlock(24, 24, 4, 1),
            ArConvBlock(24, 32, 8, 2),
            ArConvBlock(32, 32, 8, 1),
            ArConvBlock(32, 32, 8, 1),
            ArConvBlock(32, 48, 12, 2),
            ArConvBlock(48, 48, 12, 1),
            ArConvBlock(48, 48, 12, 1),
            ArConvBlock(48, 48, 12, 1),
            ArConvBlock(48, 48, 12, 1),
            ArConvBlock(48, 48, 12, 1),
            ArConvBlock(48, 48, 12, 1),
            ArConvBlock(48, 48, 12, 1),
            ArConvBlock(48, 96, 16, 2),
```

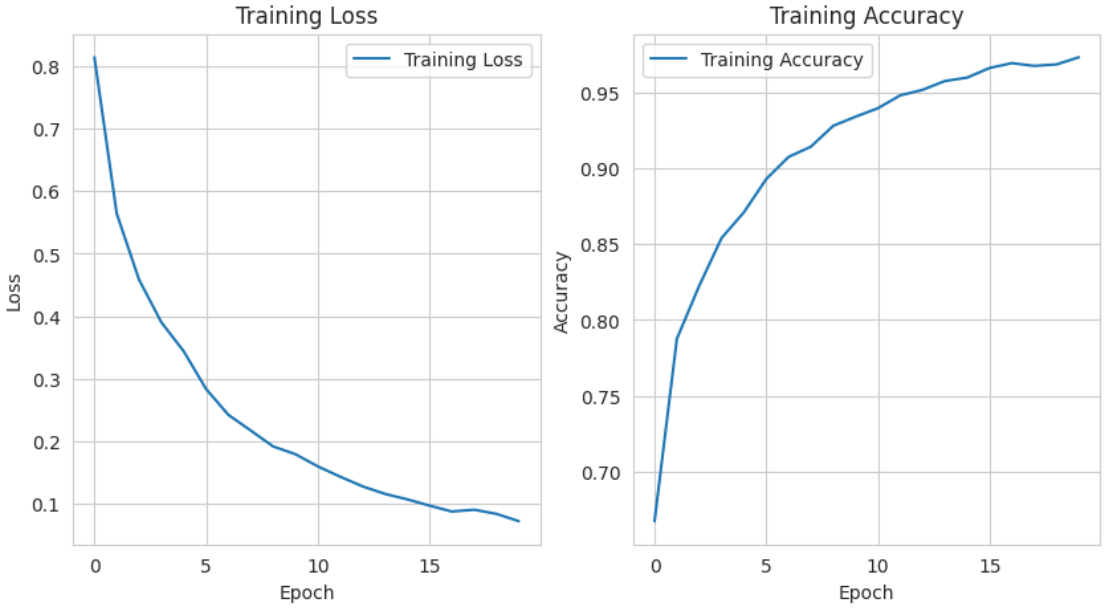
```
            ArConvBlock(96, 96, 16, 1),
            ArConvBlock(96, 96, 16, 1),
        )
        self.dense = nn.Conv2d(96, 1536, kernel_size=1)
        self.gap = nn.AdaptiveAvgPool2d(1)
        self.output = nn.Linear(1536, num_classes)

    def forward(self, x):
        x = self.initial(x)
        x = self.blocks(x)
        x = self.dense(x)
        x = self.gap(x)
        x = x.view(x.size(0), -1)
        x = self.output(x)
        return x

print("DataFrame shape:", df.shape)
print("Labels distribution:\n", df['label'].value_counts())
if df.empty:
    raise ValueError("DataFrame is empty. Please provide a valid DataFrame
with image paths and labels.")
if not all(df['label'].isin(['glioma', 'meningioma', 'no_tumor',
'pituitary'])):
    raise ValueError("DataFrame contains invalid labels. Expected: 'glioma',
'meningioma', 'no_tumor', 'pituitary'.")

valid_paths = [os.path.exists(path) for path in df['image_path']]
if not all(valid_paths):
    print("Invalid image paths found:")
    print(df.loc[~pd.Series(valid_paths), 'image_path'])
    raise ValueError("Some image paths are invalid. Please check the paths in
the DataFrame.")

merged_images_paths = []
merged_labels = []
num_merges = len(df)
temp_dir = 'merged_images'
os.makedirs(temp_dir, exist_ok=True)
groups = df.groupby('label')
resize_transform = transforms.Resize((224, 224))
for i in range(num_merges):
    label = np.random.choice(['glioma', 'meningioma', 'no_tumor',
'pituitary'])
    group = groups.get_group(label)
    idx1 = np.random.randint(len(group))
    idx2 = np.random.randint(len(group))
    img1_path = group.iloc[idx1]['image_path']
    img2_path = group.iloc[idx2]['image_path']
    try:
```

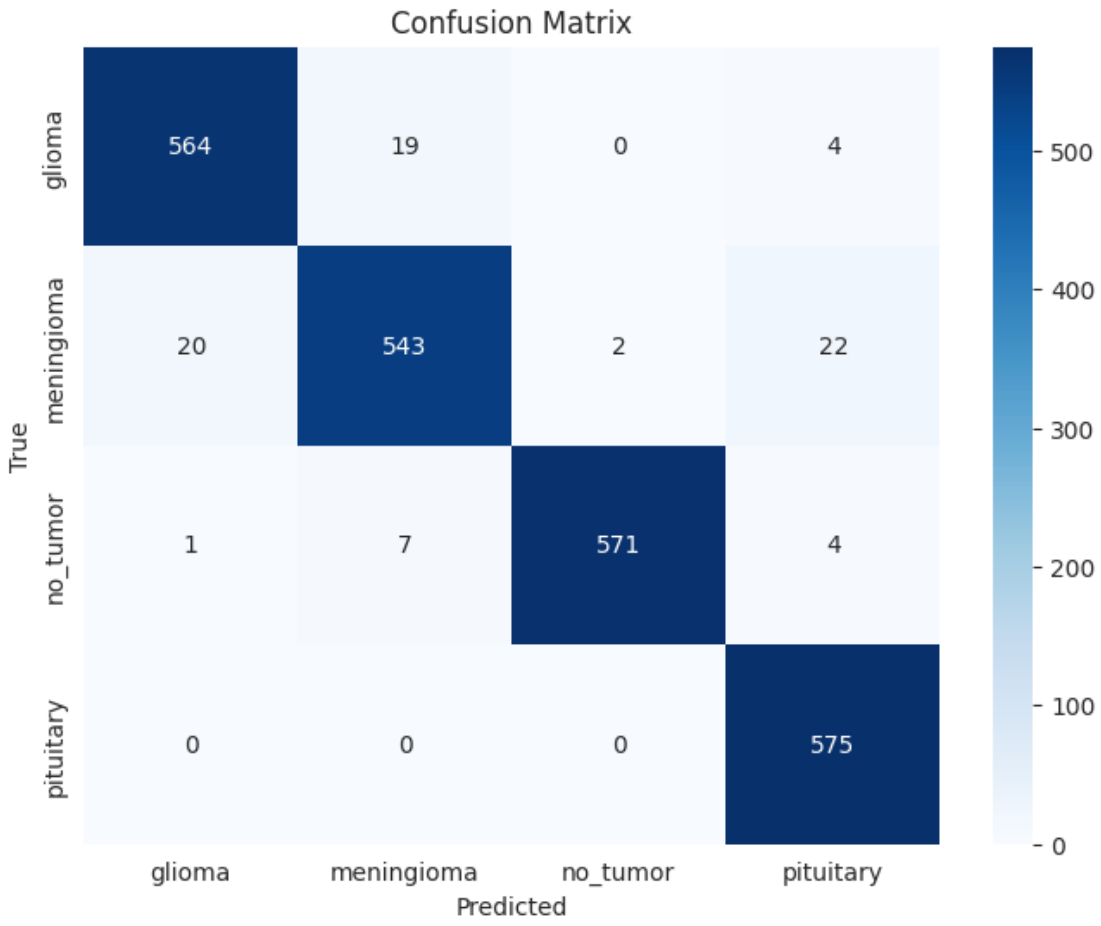
```
            img1 = Image.open(img1_path).convert('RGB')
            img2 = Image.open(img2_path).convert('RGB')
            img1 = resize_transform(img1)
            img2 = resize_transform(img2)
            img1 = np.array(img1)
            img2 = np.array(img2)
            merged = (img1.astype(float) + img2.astype(float)) / 2
            merged = merged.astype(np.uint8)
            merged_path = os.path.join(temp_dir, f'merged_{i}.jpg')
            Image.fromarray(merged).save(merged_path)
            merged_images_paths.append(merged_path)
            merged_labels.append(label)
        except Exception as e:
            print(f"Error processing images {img1_path} or {img2_path}: {e}")
            continue
merged_df = pd.DataFrame({'image_path': merged_images_paths, 'label':
merged_labels})
df = pd.concat([df, merged_df], ignore_index=True)
print(f"Total images after merging: {len(df)}")

transform = transforms.Compose([
    transforms.RandomRotation(15),
    transforms.RandomAdjustSharpness(1.2),
    transforms.RandomHorizontalFlip(),
    transforms.ColorJitter(brightness=0.2, contrast=0.2, saturation=0.2,
hue=0.1),
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224,
0.225]),
])

class BrainTumorDataset(Dataset):
    def __init__(self, df, transform=None):
        self.df = df
        self.transform = transform
        self.label_map = {'glioma': 0, 'meningioma': 1, 'no_tumor': 2,
'pituitary': 3}

    def __len__(self):
        return len(self.df)

    def __getitem__(self, idx):
        img_path = self.df.iloc[idx]['image_path']
        label_str = self.df.iloc[idx]['label']
        try:
            image = Image.open(img_path).convert('RGB')
        except Exception as e:
            print(f"Error loading image {img_path}: {e}")
```

```
            image = Image.fromarray(np.zeros((224, 224, 3), dtype=np.uint8))
        if self.transform:
            image = self.transform(image)
        label = self.label_map[label_str]
        return image, label

try:
    train_df, test_df = train_test_split(df, test_size=0.2,
stratify=df['label'], random_state=42)
    print(f"Train set size: {len(train_df)}, Test set size: {len(test_df)}")
except ValueError as e:
    print("Error during train-test split:", e)
    raise

train_dataset = BrainTumorDataset(train_df, transform=transform)
test_dataset = BrainTumorDataset(test_df, transform=transform)
train_loader = DataLoader(train_dataset, batch_size=32, shuffle=True,
num_workers=4)
test_loader = DataLoader(test_dataset, batch_size=32, shuffle=False,
num_workers=4)

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
model = ArConvNet(num_classes=4).to(device)
optimizer = optim.Adam(model.parameters(), lr=0.0001)
criterion = nn.CrossEntropyLoss()

train_losses = []
train_accuracies = []
num_epochs = 20
for epoch in range(num_epochs):
    model.train()
    running_loss = 0.0
    correct = 0
    total = 0
    for images, labels in train_loader:
        images, labels = images.to(device), labels.to(device)
        optimizer.zero_grad()
        outputs = model(images)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()
        running_loss += loss.item()
        _, predicted = torch.max(outputs.data, 1)
        total += labels.size(0)
        correct += (predicted == labels).sum().item()
    epoch_loss = running_loss / len(train_loader)
    epoch_acc = correct / total
    train_losses.append(epoch_loss)
    train_accuracies.append(epoch_acc)
```

```
    print(f'Epoch [{epoch+1}/{num_epochs}], Loss: {epoch_loss:.4f}, Accuracy:
{epoch_acc:.4f}')

plt.figure(figsize=(10, 5))
plt.subplot(1, 2, 1)
plt.plot(train_losses, label='Training Loss')
plt.title('Training Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()
plt.subplot(1, 2, 2)
plt.plot(train_accuracies, label='Training Accuracy')
plt.title('Training Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()
plt.show()

model.eval()
y_true = []
y_pred = []
with torch.no_grad():
    for images, labels in test_loader:
        images = images.to(device)
        outputs = model(images)
        _, predicted = torch.max(outputs.data, 1)
        y_true.extend(labels.cpu().numpy())
        y_pred.extend(predicted.cpu().numpy())


cm = confusion_matrix(y_true, y_pred)
plt.figure(figsize=(8, 6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=['glioma',
'meningioma', 'no_tumor', 'pituitary'], yticklabels=['glioma', 'meningioma',
'no_tumor', 'pituitary'])
plt.xlabel('Predicted')
plt.ylabel('True')
plt.title('Confusion Matrix')
plt.show()

print(classification_report(y_true, y_pred, target_names=['glioma',
'meningioma', 'no_tumor', 'pituitary']))

print(model)

DataFrame shape: (5828, 2)
Labels distribution:
 label
glioma       1457
```

```
meningioma    1457
no_tumor      1457
pituitary     1457
Name: count, dtype: int64
Total images after merging: 11656
Train set size: 9324, Test set size: 2332
Epoch [1/20], Loss: 0.8139, Accuracy: 0.6675
Epoch [2/20], Loss: 0.5639, Accuracy: 0.7876
Epoch [3/20], Loss: 0.4584, Accuracy: 0.8225
Epoch [4/20], Loss: 0.3903, Accuracy: 0.8539
Epoch [5/20], Loss: 0.3440, Accuracy: 0.8709
Epoch [6/20], Loss: 0.2838, Accuracy: 0.8927
Epoch [7/20], Loss: 0.2424, Accuracy: 0.9073
Epoch [8/20], Loss: 0.2174, Accuracy: 0.9141
Epoch [9/20], Loss: 0.1920, Accuracy: 0.9278
Epoch [10/20], Loss: 0.1796, Accuracy: 0.9338
Epoch [11/20], Loss: 0.1602, Accuracy: 0.9394
Epoch [12/20], Loss: 0.1437, Accuracy: 0.9479
Epoch [13/20], Loss: 0.1283, Accuracy: 0.9515
Epoch [14/20], Loss: 0.1163, Accuracy: 0.9573
Epoch [15/20], Loss: 0.1075, Accuracy: 0.9596
Epoch [16/20], Loss: 0.0975, Accuracy: 0.9659
Epoch [17/20], Loss: 0.0879, Accuracy: 0.9691
Epoch [18/20], Loss: 0.0908, Accuracy: 0.9673
Epoch [19/20], Loss: 0.0842, Accuracy: 0.9683
Epoch [20/20], Loss: 0.0725, Accuracy: 0.9730
```





	precision	recall	f1-score	support
glioma	0.96	0.96	0.96	587
meningioma	0.95	0.93	0.94	587
no_tumor	1.00	0.98	0.99	583
pituitary	0.95	1.00	0.97	575
accuracy			0.97	2332
macro avg	0.97	0.97	0.97	2332
weighted avg	0.97	0.97	0.97	2332

```
ArConvNet(
  (initial): Conv2d(3, 24, kernel_size=(4, 4), stride=(4, 4))
  (blocks): Sequential(
    (0): ArConvBlock(
      (expand): Conv2d(24, 96, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn1): BatchNorm2d(96, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (dw): ArConv(
        (dw_conv): Conv2d(96, 96, kernel_size=(1, 3), stride=(1, 1),
padding=(0, 1), groups=96, bias=False)
```

```
      )
      (bn2): BatchNorm2d(96, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (project): Conv2d(96, 24, kernel_size=(1, 1), stride=(1, 1),
bias=False)
      (bn3): BatchNorm2d(24, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    )
    (1): ArConvBlock(
      (expand): Conv2d(24, 96, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn1): BatchNorm2d(96, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (dw): ArConv(
        (dw_conv): Conv2d(96, 96, kernel_size=(1, 3), stride=(1, 1),
padding=(0, 1), groups=96, bias=False)
      )
      (bn2): BatchNorm2d(96, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (project): Conv2d(96, 24, kernel_size=(1, 1), stride=(1, 1),
bias=False)
      (bn3): BatchNorm2d(24, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    )
    (2): ArConvBlock(
      (expand): Conv2d(24, 96, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn1): BatchNorm2d(96, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (dw): ArConv(
        (dw_conv): Conv2d(96, 96, kernel_size=(1, 3), stride=(1, 1),
padding=(0, 1), groups=96, bias=False)
      )
      (bn2): BatchNorm2d(96, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (project): Conv2d(96, 24, kernel_size=(1, 1), stride=(1, 1),
bias=False)
      (bn3): BatchNorm2d(24, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    )
    (3): ArConvBlock(
      (expand): Conv2d(24, 192, kernel_size=(1, 1), stride=(1, 1),
bias=False)
      (bn1): BatchNorm2d(192, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (dw): Conv2d(192, 192, kernel_size=(3, 3), stride=(2, 2), padding=(1,
1), groups=192, bias=False)
      (bn2): BatchNorm2d(192, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (project): Conv2d(192, 32, kernel_size=(1, 1), stride=(1, 1),
bias=False)
      (bn3): BatchNorm2d(32, eps=1e-05, momentum=0.1, affine=True,
```

```
track_running_stats=True)
    )
    (4): ArConvBlock(
      (expand): Conv2d(32, 256, kernel_size=(1, 1), stride=(1, 1),
bias=False)
      (bn1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (dw): ArConv(
        (dw_conv): Conv2d(256, 256, kernel_size=(1, 3), stride=(1, 1),
padding=(0, 1), groups=256, bias=False)
      )
      (bn2): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (project): Conv2d(256, 32, kernel_size=(1, 1), stride=(1, 1),
bias=False)
      (bn3): BatchNorm2d(32, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    )
    (5): ArConvBlock(
      (expand): Conv2d(32, 256, kernel_size=(1, 1), stride=(1, 1),
bias=False)
      (bn1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (dw): ArConv(
        (dw_conv): Conv2d(256, 256, kernel_size=(1, 3), stride=(1, 1),
padding=(0, 1), groups=256, bias=False)
      )
      (bn2): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (project): Conv2d(256, 32, kernel_size=(1, 1), stride=(1, 1),
bias=False)
      (bn3): BatchNorm2d(32, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    )
    (6): ArConvBlock(
      (expand): Conv2d(32, 384, kernel_size=(1, 1), stride=(1, 1),
bias=False)
      (bn1): BatchNorm2d(384, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (dw): Conv2d(384, 384, kernel_size=(3, 3), stride=(2, 2), padding=(1,
1), groups=384, bias=False)
      (bn2): BatchNorm2d(384, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (project): Conv2d(384, 48, kernel_size=(1, 1), stride=(1, 1),
bias=False)
      (bn3): BatchNorm2d(48, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    )
    (7): ArConvBlock(
      (expand): Conv2d(48, 576, kernel_size=(1, 1), stride=(1, 1),
```

```
bias=False)
      (bn1): BatchNorm2d(576, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (dw): ArConv(
        (dw_conv): Conv2d(576, 576, kernel_size=(1, 3), stride=(1, 1),
padding=(0, 1), groups=576, bias=False)
      )
      (bn2): BatchNorm2d(576, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (project): Conv2d(576, 48, kernel_size=(1, 1), stride=(1, 1),
bias=False)
      (bn3): BatchNorm2d(48, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    )
    (8): ArConvBlock(
      (expand): Conv2d(48, 576, kernel_size=(1, 1), stride=(1, 1),
bias=False)
      (bn1): BatchNorm2d(576, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (dw): ArConv(
        (dw_conv): Conv2d(576, 576, kernel_size=(1, 3), stride=(1, 1),
padding=(0, 1), groups=576, bias=False)
      )
      (bn2): BatchNorm2d(576, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (project): Conv2d(576, 48, kernel_size=(1, 1), stride=(1, 1),
bias=False)
      (bn3): BatchNorm2d(48, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    )
    (9): ArConvBlock(
      (expand): Conv2d(48, 576, kernel_size=(1, 1), stride=(1, 1),
bias=False)
      (bn1): BatchNorm2d(576, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (dw): ArConv(
        (dw_conv): Conv2d(576, 576, kernel_size=(1, 3), stride=(1, 1),
padding=(0, 1), groups=576, bias=False)
      )
      (bn2): BatchNorm2d(576, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (project): Conv2d(576, 48, kernel_size=(1, 1), stride=(1, 1),
bias=False)
      (bn3): BatchNorm2d(48, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    )
    (10): ArConvBlock(
      (expand): Conv2d(48, 576, kernel_size=(1, 1), stride=(1, 1),
bias=False)
      (bn1): BatchNorm2d(576, eps=1e-05, momentum=0.1, affine=True,
```

```
track_running_stats=True)
      (dw): ArConv(
        (dw_conv): Conv2d(576, 576, kernel_size=(1, 3), stride=(1, 1),
padding=(0, 1), groups=576, bias=False)
      )
      (bn2): BatchNorm2d(576, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (project): Conv2d(576, 48, kernel_size=(1, 1), stride=(1, 1),
bias=False)
      (bn3): BatchNorm2d(48, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    )
    (11): ArConvBlock(
      (expand): Conv2d(48, 576, kernel_size=(1, 1), stride=(1, 1),
bias=False)
      (bn1): BatchNorm2d(576, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (dw): ArConv(
        (dw_conv): Conv2d(576, 576, kernel_size=(1, 3), stride=(1, 1),
padding=(0, 1), groups=576, bias=False)
      )
      (bn2): BatchNorm2d(576, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (project): Conv2d(576, 48, kernel_size=(1, 1), stride=(1, 1),
bias=False)
      (bn3): BatchNorm2d(48, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    )
    (12): ArConvBlock(
      (expand): Conv2d(48, 576, kernel_size=(1, 1), stride=(1, 1),
bias=False)
      (bn1): BatchNorm2d(576, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (dw): ArConv(
        (dw_conv): Conv2d(576, 576, kernel_size=(1, 3), stride=(1, 1),
padding=(0, 1), groups=576, bias=False)
      )
      (bn2): BatchNorm2d(576, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (project): Conv2d(576, 48, kernel_size=(1, 1), stride=(1, 1),
bias=False)
      (bn3): BatchNorm2d(48, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    )
    (13): ArConvBlock(
      (expand): Conv2d(48, 576, kernel_size=(1, 1), stride=(1, 1),
bias=False)
      (bn1): BatchNorm2d(576, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (dw): ArConv(
```

```
        (dw_conv): Conv2d(576, 576, kernel_size=(1, 3), stride=(1, 1),
padding=(0, 1), groups=576, bias=False)
      )
      (bn2): BatchNorm2d(576, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (project): Conv2d(576, 48, kernel_size=(1, 1), stride=(1, 1),
bias=False)
      (bn3): BatchNorm2d(48, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    )
    (14): ArConvBlock(
      (expand): Conv2d(48, 768, kernel_size=(1, 1), stride=(1, 1),
bias=False)
      (bn1): BatchNorm2d(768, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (dw): Conv2d(768, 768, kernel_size=(3, 3), stride=(2, 2), padding=(1,
1), groups=768, bias=False)
      (bn2): BatchNorm2d(768, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (project): Conv2d(768, 96, kernel_size=(1, 1), stride=(1, 1),
bias=False)
      (bn3): BatchNorm2d(96, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    )
    (15): ArConvBlock(
      (expand): Conv2d(96, 1536, kernel_size=(1, 1), stride=(1, 1),
bias=False)
      (bn1): BatchNorm2d(1536, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (dw): ArConv(
        (dw_conv): Conv2d(1536, 1536, kernel_size=(1, 3), stride=(1, 1),
padding=(0, 1), groups=1536, bias=False)
      )
      (bn2): BatchNorm2d(1536, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (project): Conv2d(1536, 96, kernel_size=(1, 1), stride=(1, 1),
bias=False)
      (bn3): BatchNorm2d(96, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    )
    (16): ArConvBlock(
      (expand): Conv2d(96, 1536, kernel_size=(1, 1), stride=(1, 1),
bias=False)
      (bn1): BatchNorm2d(1536, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
      (dw): ArConv(
        (dw_conv): Conv2d(1536, 1536, kernel_size=(1, 3), stride=(1, 1),
padding=(0, 1), groups=1536, bias=False)
      )
      (bn2): BatchNorm2d(1536, eps=1e-05, momentum=0.1, affine=True,
```

```
track_running_stats=True)
      (project): Conv2d(1536, 96, kernel_size=(1, 1), stride=(1, 1),
bias=False)
      (bn3): BatchNorm2d(96, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    )
  )
  (dense): Conv2d(96, 1536, kernel_size=(1, 1), stride=(1, 1))
  (gap): AdaptiveAvgPool2d(output_size=1)
  (output): Linear(in_features=1536, out_features=4, bias=True)
)
```